Web Design Foundations

HTML & CSS – Part 1

Seyed Ahmad Mousavi Zadeh

Structure, style, layout, and interaction-ready interfaces

A visual, example-driven course

Learning outcomes

By the end of this part, learners will be able to:

- Explain web design and the roles of front end and back end.
- Create a valid HTML document and use common semantic and content elements.
- Apply CSS through inline, internal, and external stylesheets.
- Style typography, color, borders, spacing, tables, forms, and media.
- Build layouts with positioning, Flexbox, and CSS Grid.
- Add responsive behavior, transitions, animations, shadows, gradients, and custom fonts.

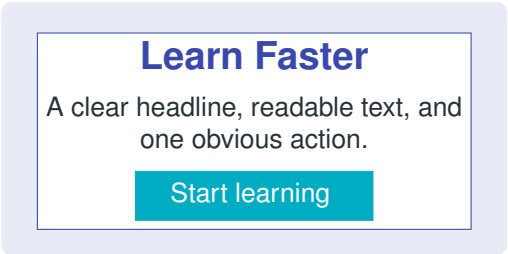
Scope

JavaScript is intentionally reserved for Part 2. This part focuses on HTML structure and CSS presentation.

What is web design?

Web design is the planning and creation of a website's structure, appearance, content, and user experience.

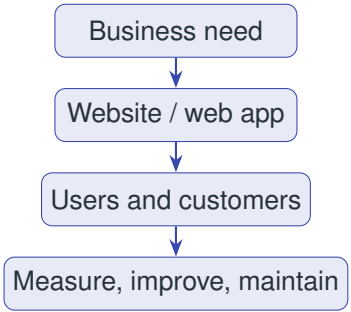
- **Usability:** visitors can complete tasks.
- **Visual design:** color, type, spacing, hierarchy.
- **Accessibility:** people with different abilities can use it.
- **Responsive design:** layouts adapt to screens.
- **Performance:** pages load and respond efficiently.



Why is web development in demand?

- Businesses need websites, online stores, portals, and internal tools.
- Every industry needs a digital presence and ongoing maintenance.
- Front-end skills connect design with usable, accessible interfaces.
- Skills can lead to employment, freelancing, entrepreneurship, or further study.

Demand varies by region and economic conditions. A portfolio, problem-solving, communication, and continuous learning matter alongside technical skills.



Front end vs. back end

Front end – in the browser

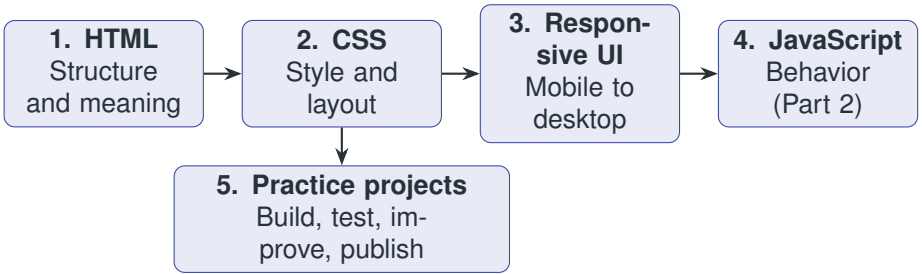
- What users see and interact with
- HTML: structure
- CSS: presentation and layout
- JavaScript: behavior (Part 2)

Back end – on the server

- Business rules and data processing
- Authentication and permissions
- Databases and APIs
- Server-side languages and infrastructure



The front-end learning roadmap



Our route in this booklet

HTML foundations and elements → CSS syntax and selectors → styling → layout → responsive design and effects.

How a web page is made

1. Create a project folder, for example `my-site/`.
2. Create `index.html` for the page structure.
3. Create `style.css` for visual styling.
4. Open `index.html` in a browser.
5. Edit, save, refresh, and inspect the result.

Recommended tools

A code editor, a modern browser, and browser Developer Tools. Keep filenames simple and use relative paths carefully.

The minimum HTML document

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport"
        content="width=device-width, initial-scale=1">
  <title>My First Page</title>
</head>
<body>
  <h1>Hello, web!</h1>
  <p>This is my first page.</p>
</body>
</html>
```

Browser result

Hello, web!

This is my first page.

The doctype enables standards mode. The head stores metadata; the body contains visible page content.

Elements, tags, attributes

- An **element** commonly has an opening tag, content, and a closing tag.
- An **attribute** adds information to an element.
- Some elements are void elements and do not have closing tags, such as `img`, `br`, `hr`, and `input`.

```
<a href="https://example.com"
   target="_blank" rel="noopener">
  Visit example
</a>
```

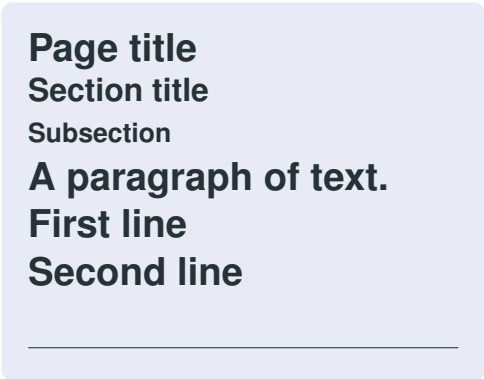
Browser result

Visit example The link opens the destination in a new tab.

The href attribute sets the destination. Use `rel="noopener"` with `target="_blank"` as a security best practice.

Paragraphs, headings, breaks, and rules

```
<h1>Page title</h1>
<h2>Section title</h2>
<h3>Subsection</h3>
<p>A paragraph of text.</p>
<p>First line<br>Second line</p>
<hr>
```



Headings range from `h1` to `h6`. Choose them by document hierarchy, not merely by visual size. `center` is obsolete in HTML5; use CSS `text-align: center`.

Text emphasis and special formatting

```
<p><i>Italic appearance</i></p>
<p><em>Emphasis in meaning</em></p>
<p><u>Underlined text</u></p>
<p><ins>Inserted text</ins></p>
<p>H<sub>2</sub>O and x<sup>2</sup></p>
<p><b>Bold appearance</b></p>
<p><strong>Important text</strong></p>
<p><small>Small print</small></p>
```

Italic appearance
Emphasis in meaning
Underlined text
Inserted text
H_2O and x^2
Bold appearance
Important text
Small print

Use `em` and `strong` for meaning; use CSS for visual styling. Underlines can be confused with links.

Preformatted text and whitespace

```
<pre>
Name        Score
Ada         98
Lin         95
</pre>
```

```
Name    Score
Ada     98
Lin     95
```

Important

`pre` preserves spaces and line breaks. Ordinary HTML collapses repeated whitespace. Use `code` inside `pre` when showing source code.

Links and bookmarks

```
<h2 id="courses">Courses</h2>
<a href="#courses">Jump to Courses</a>
<a href="contact.html">Contact</a>
<a href="https://example.com"
   target="_blank"
   rel="noopener">External site</a>
```

Courses

Jump to Courses
Contact
External site

- A bookmark is an in-page link to an element with a unique `id`.
- `href="#courses"` moves the browser to that section.
- For a downloadable or external resource, provide a meaningful link label.

Lists: ul, ol, li, and dl

```
<ul><li>HTML</li><li>CSS</li></ul>
<ol><li>Plan</li><li>Build</li></ol>
<dl>
  <dt>HTML</dt>
  <dd>Page structure</dd>
</dl>
```

Unordered
- HTML
- CSS

Ordered
1. Plan
2. Build

Description
HTML – Page structure

`ul` creates a bulleted list, `ol` a numbered list, `li` a list item, and `dl` a description list containing `dt` terms and `dd` descriptions.

Images and accessible alternatives

```
<img src="images/campus.jpg"
     alt="Students walking on campus"
     width="280">
```

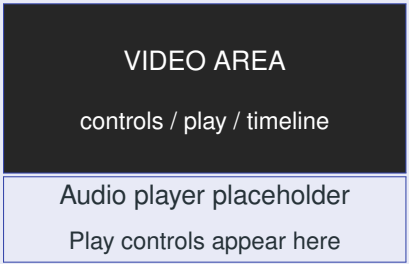


- `src` is the image path; `alt` describes meaningful image content.
- Use descriptive filenames and appropriately sized images.
- Decorative images can use `alt=""`.

Audio and video

```
<video controls width="320"
       muted autoplay loop
       preload="metadata">
  <source src="media/demo.mp4"
          type="video/mp4">
  Your browser cannot play this video.
</video>

<audio controls preload="none">
  <source src="media/theme.mp3"
          type="audio/mpeg">
</audio>
```



`autoplay` may be blocked unless muted. `loop` repeats playback. `preload` accepts `none`, `metadata`, or `auto`; it is a browser hint, not a guarantee.

span and display: none

```
<p>Learn <span class="highlight">
HTML</span> today.</p>
```

```
.highlight { color: #3949AB;
             font-weight: 700; }
.hidden { display: none; }
```

Learn **HTML** today.

An element with `display: none` is not rendered and takes no layout space.

Remember

`span` is an inline container with no meaning by itself. `display: none` removes an element from rendering; it is a CSS value, not an HTML tag.

Tables: rows, cells, and sections

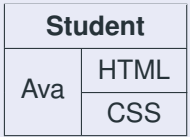
```
<table>
  <thead>
    <tr><th>Course</th><th>Hours</th></tr>
  </thead>
  <tbody>
    <tr><td>HTML</td><td>4</td></tr>
    <tr><td>CSS</td><td>6</td></tr>
  </tbody>
  <tfoot>
    <tr><th>Total</th><td>10</td></tr>
  </tfoot>
</table>
```

Course	Hours
HTML	4
CSS	6
Total	10

`table` contains rows (`tr`); `th` labels a header cell and `td` a data cell. Use `thead`, `tbody`, and `tfoot` to group rows. Tables are for tabular data, not page layout.

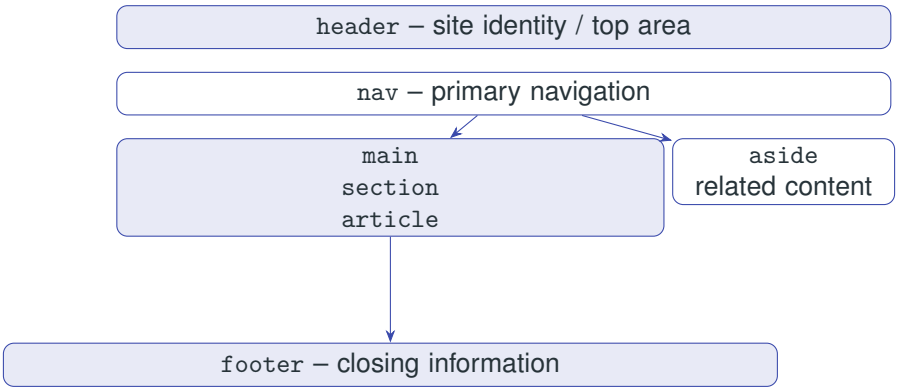
Table spanning: colspan and rowspan

```
<table>
 <tr><th colspan="2">Student</th></tr>
 <tr><td rowspan="2">Ava</td>
     <td>HTML</td></tr>
 <tr><td>CSS</td></tr>
</table>
```



`colspan` makes a cell cover multiple columns; `rowspan` makes it cover multiple rows. Check the number of cells in each row carefully.

Semantic page structure



`article` = self-contained composition (a blog post, a card). `section` = thematic grouping inside an article/main. `div` = generic block container when nothing more meaningful fits.

div, figure, and figcaption

Code

```
<div class="card">
  <figure>
    <img src="chart.png"
        alt="Sales chart">
    <figcaption>
        Sales increased.
    </figcaption>
  </figure>
</div>

<article>
  <h2>HTML Basics</h2>
  <section>
    <h3>Tags</h3>
    <p>...</p>
  </section>

</article>
```

`figure` groups media; `figcaption` = caption. `article` = self-contained; `section` = thematic group.



CSS: what it does

CSS (Cascading Style Sheets) controls how HTML is presented: color, typography, spacing, borders, positioning, and responsive layout.

```
h1 {
  color: #3949AB;
  background: #E8EAF6;
  padding: 12px;
  border-radius: 8px;
}
```



A CSS rule is `selector { property: value; }`. The selector chooses elements; declarations define their appearance. Without CSS, the same `h1` would render as plain black text.

Three ways to write CSS

Inline

```
<p style="color: indigo">
  Hello
</p>
```

Hello

Internal

```
<style>
p { color: indigo; }
</style>
<p>Hello</p>
```

Hello

External

```
<link rel="stylesheet"
      href="style.css">
<p>Hello</p>
```

Hello

Recommendation

Use an external stylesheet for most projects: it separates content from presentation and is reusable across pages.

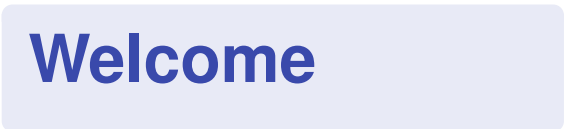
External CSS: linking and file structure

index.html

```
<head>
  <link rel="stylesheet"
        href="css/style.css">
</head>
<body>
  <h1>Welcome</h1>
</body>
```

css/style.css

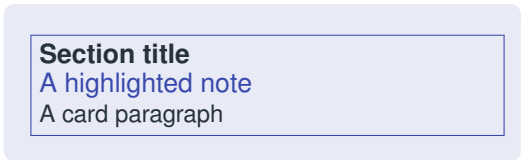
```
h1 {
  color: #3949AB;
  font-size: 2rem;
}
```



The path in `href` must match the actual file location. CSS comments use `/* ... */`.

Selectors and the cascade

```
p { color: #263238; }       /* element */
.note { color: #3949AB; }   /* class */
#hero { color: #1A237E; }   /* id */
h1, h2 { font-family: sans-serif; }
.card p { margin: 0; }      /* descendant */
* { box-sizing: border-box; }
```



Specificity: inline ≻ `#id` ≻ `.class` ≻ `element` ≻ `*`. Source order and `!important` also affect which rule wins. Prefer classes for reusable styling.

CSS Units: px, %, em, rem, vw, vh

```
.a { font-size: 16px; }        /* absolute */
.b { width: 60%; }             /* parent */
.c { padding: 1em; }           /* parent font */
.d { font-size: 1.5rem; }      /* root font */
.e { width: 80vw; height: 50vh; }
.f { font-size: clamp(1rem, 2vw, 2rem); }
```

Absolute
always same size

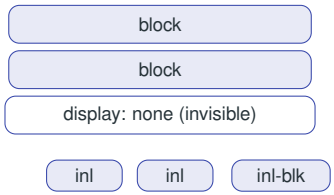
Relative
scales with context

Rule of thumb

Use `rem` for font sizes, `px` for fine borders, `%`/`vw`/`vh` for layout, and `clamp()` for fluid typography.

display: block, inline, inline-block, none

```
.a { display: block; }
.b { display: inline; }
.c { display: inline-block; }
.d { display: none; }
```



- **block**: full width, new line. **inline**: content width, no top/bottom margin. **inline-block**: in-line but accepts width/height. **none**: removed from layout.

Color formats and standards

```
.a { color: indigo; }              /* named */
.b { color: #3949AB; }             /* HEX */
.c { color: rgb(57,73,171); }      /* RGB */
.d { color: rgba(57,73,171,.7); } /* alpha */
.e { color: hsl(232,50%,45%); }    /* HSL */
.f { color: hwb(232 22% 33%); }    /* HWB */
```

Indigo



CSS supports named colors, hexadecimal, RGB/RGBA, HSL/HSLA, and modern formats such as HWB and OKLCH in current browsers. Alpha controls transparency. Prefer sufficient text/background contrast.

Borders, radius, and collapse

```
.card {
  border: 2px solid #3949AB;
  border-radius: 14px;
  padding: 12px;
}
table {
  border-collapse: collapse;
}
td, th {
  border: 1px solid #9FA8DA;
}
```

Rounded card
A clear border and soft corners.

A	B
1	2

`border` is a shorthand for width, style, and color. `border-collapse: collapse` merges adjacent table borders.

Text alignment, color, and background

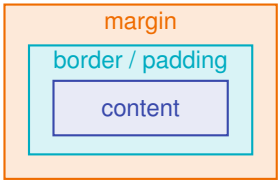
```
.notice {
  color: #1A237E;
  background-color: #E8EAF6;
  text-align: center;
  padding: 12px;
}
```

Centered notice
Indigo text on a pale background.

- `text-align`: `left`, `right`, `center`, `justify`.
- `color` sets foreground text color; `background-color` sets the background color.

Margin, padding, and the box model

```
.box {
  margin: 18px;
  padding: 16px;
  border: 4px solid #3949AB;
  width: 220px;
  box-sizing: border-box;
}
```



Margin is outside the border; padding is inside it. `content-box` adds padding and border to declared width; `border-box` includes them.

Lists and background images

```
ul {
  list-style-type: square;
}
.custom-list {
  list-style-image: url("star.png");
}
.hero {
  background-image:
    linear-gradient(rgba(26,35,126,.7),
                    rgba(26,35,126,.7)),
    url("hero.jpg");
  background-size: cover;
  background-position: center;
}
```

Topics

- HTML
- CSS

Hero background

image + overlay

Use `list-style-image` only when the image is appropriate. A gradient overlay improves text contrast over a background image.

Direction, decoration, and text transforms

```
.rtl {
  direction: rtl;
}
.link {
  text-decoration: underline;
}
.label {
  text-transform: uppercase;
}
```

RTL example

Hello World

Underlined link

COURSE LABEL

`direction` affects text direction and some layout behavior. Prefer the HTML `dir` attribute when direction is part of the content, e.g., `<html lang="fa" dir="rtl">`.

Typography properties

```
.sample {
  font-family: Arial, sans-serif;
  font-weight: 700;
  font-size: 20px;
  font-style: italic;
  text-indent: 1.5em;
  letter-spacing: 1px;
  line-height: 1.6;
  word-spacing: 4px;
}
```

Typography sample

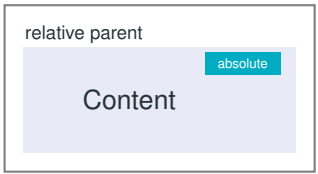
A readable line height gives text room to breathe.

Letter and word spacing are exaggerated here.

- Use `rem` for scalable font-size; `1.5–1.7` line-height for body text.
- `font-weight` ranges 100–900 when the font supports it.

Positioning: static, relative, absolute

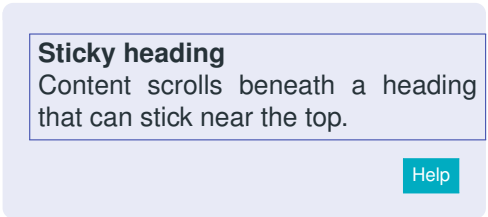
```
.static  { position: static; }
.relative { position: relative; top: 8px; }
.parent { position: relative; }
.badge {
  position: absolute;
  top: 8px; right: 8px;
}
```



`static`: default flow. `relative`: offset from normal position. `absolute`: removed from flow, positioned to nearest positioned ancestor.

Positioning: fixed and sticky

```
.help-button {
  position: fixed;
  right: 18px;
  bottom: 18px;
}
.sticky-title {
  position: sticky;
  top: 0;
}
```



`fixed` stays visible while scrolling. `sticky` behaves normally until a threshold. `z-index` controls stacking in positioned elements.

Transform: translate, rotate, scale, skew

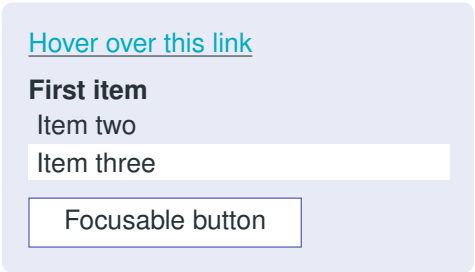
```
.t1 { transform: translateX(20px); }
.t2 { transform: rotate(45deg); }
.t3 { transform: scale(1.3); }
.t4 { transform: skewX(15deg); }
.t5 {
  transform: translate(10px,5px)
             rotate(15deg)
             scale(1.1);
}
```



Transform does not affect surrounding layout; it applies visually. Often paired with `transition` for smooth motion.

Pseudo-classes

```
a:hover { color: #00ACC1; }
a:visited { color: #5E35B1; }
button:focus-visible {
  outline: 3px solid orange;
}
input:checked + label {
  color: #2E7D32;
}
li:first-child { font-weight: 700; }
tr:nth-child(even) {
  background: #E8EAF6;
}
```



Pseudo-classes select elements in a state or position: `:hover`, `:focus-visible`, `:checked`, `:first-child`, `:nth-child()`.

Pseudo-elements

```
.tip::before {
  content: "TIP: ";
  color: #3949AB;
  font-weight: 700;
}
p::first-letter {
  font-size: 2em;
  color: #3949AB;
}
::selection {
  background: #C5CAE9;
}
```

TIP: Use meaningful HTML.

A pseudo-element can style part of an element.

Selected text preview

Pseudo-elements style a part or generate decorative content. Use `::before`/`::after` for decoration, not essential information.

details and summary

```
<details>
  <summary>What is CSS?</summary>
  CSS describes how HTML is presented.
</details>
```

> **What is CSS?**
>
> CSS describes how HTML is presented.

Why use them?

`details` creates a disclosure widget; `summary` supplies its visible label. The browser provides basic open/close behavior without JavaScript.

Forms: labels, inputs, and buttons

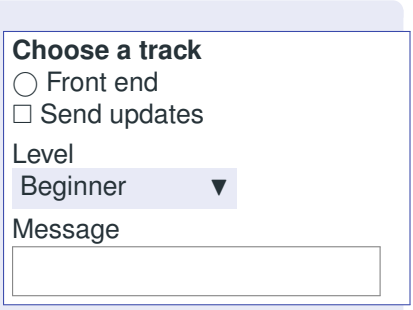
```
<form action="/signup" method="post">
  <label for="email">Email</label>
  <input id="email" name="email"
         type="email" required>
  <button type="submit">Join</button>
</form>
```



- Connect `label for` to the input's `id`.
- Choose the correct input `type`; use `name` for submitted data.
- `required` adds browser validation. A real server is still needed to process data.

More form controls

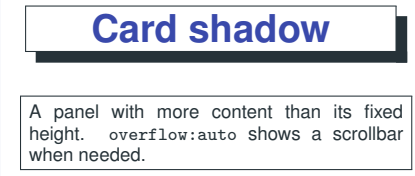
```
<fieldset>
  <legend>Choose a track</legend>
  <label><input type="radio" name="track">
    Front end</label>
  <label><input type="checkbox" name="news">
    Send updates</label>
  <label for="level">Level</label>
  <select id="level" name="level">
    <option>Beginner</option>
    <option>Intermediate</option>
  </select>
  <label for="message">Message</label>
  <textarea id="message" rows="3"></textarea>
</fieldset>
```



`fieldset` groups related controls; `legend` names the group. Also explore `password`, `number`, `date`, `radio`, `checkbox`.

Shadows and overflow

```
.card {
  box-shadow: 0 8px 20px #1A237E22;
}
.title {
  text-shadow: 2px 2px 3px #9FA8DA;
}
.panel {
  width: 180px;
  height: 70px;
  overflow: auto;
}
```



`overflow`: `visible`, `hidden`, `clip`, `scroll`, `auto`. Avoid hiding important content.

Linear gradients

```
.banner {
  background: linear-gradient(
    135deg,
    #1A237E 0%,
    #3949AB 55%,
    #00ACC1 100%
  );
  color: white;
}
```



A linear gradient blends colors along a line. First argument sets direction; color stops control where each color appears. Gradients can be layered with background images.

Transitions

```
.button {
  background: #3949AB;
  color: white;
  transform: translateY(0);
  transition: background-color .25s ease,
              transform .25s ease;
}
.button:hover {
  background: #00ACC1;
  transform: translateY(-3px);
}
```

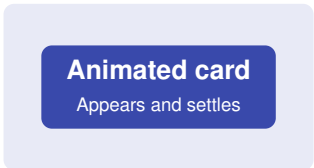
Hover-style button

Move the pointer over a real button in the browser to see the transition.

- Transitions interpolate property values when a state changes.
- Common parts: property, duration, timing function, delay.
- Prefer animating `transform` and `opacity` for smoother effects.

Animations with @keyframes

```
@keyframes float-in {
  0%   { transform: translateY(8px);
         opacity: 0; }
  100% { transform: translateY(0);
         opacity: 1; }
}
.card {
  animation: float-in .7s ease-out both;
}
```



- Shorthand: name, duration, timing, delay, iterations, direction, fill-mode, play-state.
- Useful values: `infinite`, `alternate`, `forwards`, `paused`.
- Respect reduced-motion preferences; motion should support, not distract.

Motion accessibility and reduced motion

```
@media (prefers-reduced-motion: reduce) {
  *,
  *::before,
  *::after {
    scroll-behavior: auto !important;
    animation-duration: 0.01ms !important;
    animation-iteration-count: 1 !important;
    transition-duration: 0.01ms !important;
  }
}
```

Good practice

Avoid flashing effects and endless motion when it is not necessary. Provide clear focus states and make interactive controls understandable without animation.

Flexbox: one-dimensional layout

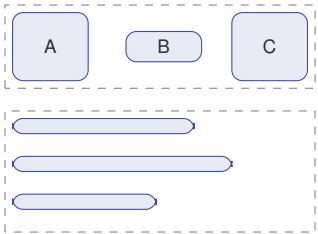
```
.container {
  display: flex;
  flex-direction: row;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  flex-wrap: wrap;
}
.item { flex: 1 1 140px; }
```



- Main axis: `row` or `column` (plus reverse).
- `justify-content`: `flex-start`, `flex-end`, `center`, `space-between`, `space-around`, `space-evenly`.
- `align-items`: `stretch`, `flex-start`, `flex-end`, `center`, `baseline`.
- Items: `order`, `flex-grow`, `flex-shrink`, `flex-basis`, `align-self`.

Flexbox: alignment in action

```
.row {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
.col {
  display: flex;
  flex-direction: column;
  justify-content: center;
  align-items: flex-start;
  gap: .5rem;
}
```



`justify-content` aligns along the main axis; `align-items` aligns across it. `flex-wrap` lets items move to another line; `align-content` distributes the lines.

CSS Grid: rows and columns

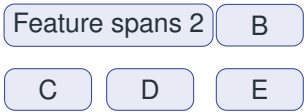
```
.layout {
  display: grid;
  grid-template-columns: 2fr 1fr;
  grid-template-rows: auto 1fr auto;
  gap: 12px;
}
.header, .footer { grid-column: 1 / -1; }
```



- `fr` distributes free space; `repeat()`, `minmax()`, `auto-fit`/`auto-fill` build flexible tracks.
- `grid-template-areas` names regions for readable layouts.

CSS Grid: placement and alignment

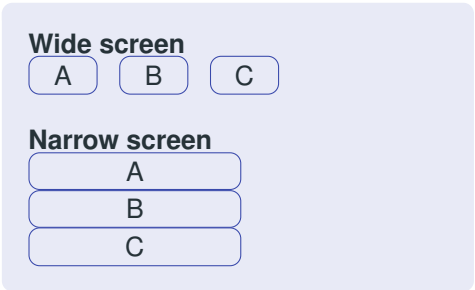
```
.grid {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 10px;
  justify-items: stretch;
  align-items: center;
}
.feature {
  grid-column: span 2;
  grid-row: span 1;
}
```



Alignment: `justify-items`, `align-items`, `place-items`, `justify-content`, `align-content`, `place-content`. Items can be placed with line numbers, named areas, or spans.

Responsive design and @media

```
.cards {
  display: grid;
  grid-template-columns: repeat(3, 1fr);
  gap: 1rem;
}
@media screen and (max-width: 700px) {
  .cards {
    grid-template-columns: 1fr;
  }
}
```



Correct syntax: `@media screen and (...)` – not `@media-screen`. Choose breakpoints where content needs them, not for specific devices.

Custom fonts with @font-face

```
@font-face {
  font-family: "Course Sans";
  src: url("fonts/course-sans.woff2")
       format("woff2");
  font-weight: 400;
  font-style: normal;
  font-display: swap;
}
body {
  font-family: "Course Sans", sans-serif;
}
```

Aa Bb Cc 123

Custom font preview

The actual appearance depends on the font file.

- Use licensed font files and provide fallback fonts.
- `font-display: swap` allows fallback text while the font loads.
- Match declared weight/style to the supplied font files.

CSS custom properties and :root

```
:root {
  --brand: #3949AB;
  --surface: #E8EAF6;
  --radius: 14px;
}
.card {
  color: var(--brand);
  background: var(--surface);
  border-radius: var(--radius);
}
```

Reusable design token

Change one variable to update many components.

`:root` selects the document root (the `html` element). Custom properties declared there are available throughout the page and can be overridden in a smaller scope.

Favicon: the small site icon

```
<head>
  <link rel="icon" href="/favicon.ico">
  <link rel="icon" type="image/png"
        href="/favicon-32.png"
        sizes="32x32">
</head>
```



Place favicon links in the document head. Keep the icon recognizable at small sizes; browsers may cache favicons.

A compact complete page

```
<!doctype html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1">
  <title>Learning Hub</title>
  <link rel="icon" href="favicon.ico">
  <link rel="stylesheet" href="style.css">
</head>
<body>
  <header><h1>Learning Hub</h1></header>
  <main>
    <section class="card">
      <h2>Start with HTML</h2>
      <p>Build meaningful page structure.</p>
      <a href="#next">Explore next</a>
    </section>
    <section id="next"><h2>Then learn CSS</h2></section>
  </main>
  <footer><small>Practice makes progress.</small></footer>
</body>
</html>
```

A compact stylesheet for the page

```
:root {
  --brand: #3949AB;
  --surface: #E8EAF6;
}
* { box-sizing: border-box; }
body {
  margin: 0;
  font-family: Arial, sans-serif;
  line-height: 1.6;
}
header, footer {
  padding: 1rem;
  background: var(--brand);
  color: white;
}
main { max-width: 900px; margin: auto; padding: 1rem; }
.card {
  padding: 1.25rem;
  border-radius: 14px;
  background: var(--surface);
  box-shadow: 0 8px 20px #1A237E22;
}
a:hover { color: #00838F; }
@media screen and (max-width: 600px) {
  main { padding: .6rem; }
}
```

Learning Hub

Start with HTML
Build meaningful page structure.

Explore next

Footer: Practice makes progress.

Common beginner mistakes

- Forgetting closing tags, document metadata, or the viewport meta tag.
- Using headings only to make text large instead of to show hierarchy.
- Using tables for page layout rather than Flexbox or Grid.
- Forgetting `alt` text, form labels, keyboard focus, or color contrast.
- Linking the wrong CSS path or using inconsistent filenames.
- Overusing absolute positioning or fixed pixel widths.
- Adding motion without considering reduced-motion preferences.
- Treating a browser-only form as if it securely stores or sends data.

Practice lab: build a course card

Task: Build a responsive course card using only HTML and CSS.

1. Add a semantic `article`, heading, paragraph, image, and link.
2. Style the card with a border, radius, padding, and box shadow.
3. Use Flexbox or Grid to arrange the content.
4. Add a hover transition and a visible keyboard focus state.
5. At a narrow viewport, change the layout with a media query.
6. Test the page with keyboard navigation and inspect it in DevTools.

Success criteria

The page is readable, responsive, accessible, and uses separate HTML and CSS files.

Recap and next lesson

Part 1 – covered

- HTML document structure and elements
- Links, lists, media, tables, and forms
- CSS syntax, selectors, typography, and color
- Positioning, pseudo-selectors, effects
- Flexbox, Grid, responsive CSS

Part 2 – next

- JavaScript fundamentals
- Variables, types, conditions, loops
- Functions and events
- DOM selection and manipulation
- Form interaction and small projects

Structure first. Style second. Behavior next.